

```

import java.io.IOException;
import java.nio.file.*;
import java.util.*;

public class DocMatching {

    private static final double MIN_SIMILARITY_THRESHOLD =
30.0;

    // Helper class to store pre-processed file data
    static class DocData {
        String fullText;
        List<String> chunks;

        DocData(String fullText, List<String> chunks) {
            this.fullText = fullText;
            this.chunks = chunks;
        }
    }

    static class MatchResult {
        String file1;
        String file2;
        double similarity;

        MatchResult(String file1, String file2, double
similarity) {
            this.file1 = file1;
            this.file2 = file2;
            this.similarity = similarity;
        }
    }

    public static void main(String[] args) {
        Path corpusDir = Paths.get("corpus");

        if (!Files.exists(corpusDir) || !
Files.isDirectory(corpusDir)) {
            System.out.println("Error: 'corpus' folder not
found.");
            return;
        }

        try {
            // 1. Read all .txt files from corpus folder
            List<Path> filePaths = new ArrayList<>();
            try (DirectoryStream<Path> stream =
Files.newDirectoryStream(corpusDir, "*.txt")) {
                for (Path entry : stream) {

```

```
        filePaths.add(entry);
    }
}
```

```
    if (filePaths.size() < 2) {
        System.out.println("Error: Need at least 2
files to compare.");
        return;
    }
}
```

```
    System.out.println("Reading and processing " +
filePaths.size() + " files in the background...");
```

```
    // 2. Pre-process text and sentence chunks for ALL
files just ONCE
    Map<String, DocData> fileDataMap = new
HashMap<>();
    for (Path path : filePaths) {
        fileDataMap.put(path.getFileName().toString(),
processFile(path));
    }
```

```
    List<String> fileNames = new
ArrayList<>(fileDataMap.keySet());
    List<MatchResult> results = new ArrayList<>();
```

```
    // 3. Compare every pair of files using Z-
Algorithm
    for (int i = 0; i < fileNames.size(); i++) {
        for (int j = i + 1; j < fileNames.size(); j++)
        {
            String file1 = fileNames.get(i);
            String file2 = fileNames.get(j);
```

```
            DocData doc1 = fileDataMap.get(file1);
            DocData doc2 = fileDataMap.get(file2);
```

```
            double similarity =
calculateSimilarityZ(doc1, doc2);
```

```
            if (similarity >=
MIN_SIMILARITY_THRESHOLD) {
                results.add(new MatchResult(file1,
file2, similarity));
            }
        }
    }
```

```
    // 4. Sort from highest similarity to lowest
```

```
        results.sort((a, b) ->
Double.compare(b.similarity, a.similarity));
```

```
        // 5. Display results
        System.out.println("\nComparisons complete.
Showing similar files (Highest to Lowest):\n");
```

```
System.out.println("-----
-----");
```

```
        if (results.isEmpty()) {
            System.out.println("No files found with a
similarity above " + MIN_SIMILARITY_THRESHOLD + "%.");
        } else {
            for (MatchResult result : results) {
                System.out.printf("Pair: %s <--> %s\n",
result.file1, result.file2);
                System.out.printf("Similarity: %.2f%%\n",
result.similarity);
```

```
                if (result.similarity >= 80.0) {
                    System.out.println("Verdict: 🚨 HIGH
CHANCE OF COPYING (Nearly identical)");
                } else if (result.similarity >= 50.0) {
                    System.out.println("Verdict: ⚠️
SUSPICIOUSLY SIMILAR (Heavy overlap)");
                } else {
                    System.out.println("Verdict: 👁️
RELATED (Some shared content)");
                }
            }
        }
```

```
System.out.println("-----
-----");
    }
}
```

```
    } catch (IOException e) {
        System.out.println("Error reading files: " +
e.getMessage());
    }
}
```

```
    // Calculates similarity by checking how many sentences
from Doc1 are in Doc2 (and vice versa)
    private static double calculateSimilarityZ(DocData doc1,
DocData doc2) {
        if (doc1.chunks.isEmpty() && doc2.chunks.isEmpty())
return 100.0;
```

```
        if (doc1.chunks.isEmpty() || doc2.chunks.isEmpty())  
return 0.0;
```

```
int matchedChunks = 0;
```

```
// Check doc1 sentences in doc2 text  
for (String chunk : doc1.chunks) {  
    if (zAlgorithmContains(doc2.fullText, chunk)) {  
        matchedChunks++;  
    }  
}
```

```
// Check doc2 sentences in doc1 text  
for (String chunk : doc2.chunks) {  
    if (zAlgorithmContains(doc1.fullText, chunk)) {  
        matchedChunks++;  
    }  
}
```

```
int totalChunks = doc1.chunks.size() +  
doc2.chunks.size();  
return ((double) matchedChunks / totalChunks) * 100.0;  
}
```

```
// True linear-time Z-Algorithm for exact substring  
matching  
private static boolean zAlgorithmContains(String text,  
String pattern) {  
    if (pattern == null || text == null ||  
pattern.isEmpty() || text.isEmpty()) {  
        return false;  
    }  
}
```

```
String concat = pattern + "$" + text;  
int n = concat.length();  
int[] Z = new int[n];  
int l = 0, r = 0;  
int patternLength = pattern.length();
```

```
for (int i = 1; i < n; i++) {  
    if (i > r) {  
        l = r = i;  
        while (r < n && concat.charAt(r) ==  
concat.charAt(r - l)) {  
            r++;  
        }  
        Z[i] = r - l;  
        r--;  
    } else {
```

```

        int k = i - l;
        if (Z[k] < r - i + 1) {
            Z[i] = Z[k];
        } else {
            l = i;
            while (r < n && concat.charAt(r) ==
concat.charAt(r - l)) {
                r++;
            }
            Z[i] = r - l;
            r--;
        }
    }

    // If the Z-value matches pattern length, the
chunk was found in the text
    if (Z[i] == patternLength) {
        return true;
    }
}
return false;
}

```

```

// Reads a file, stores the full lowercase text, and
breaks it into valid sentence chunks
private static DocData processFile(Path path) throws
IOException {
    String text = Files.readString(path).toLowerCase();

    // Break into chunks by punctuation and newlines
    String[] rawChunks = text.split("(?<=[.!?\\n])\\s+");
    List<String> validChunks = new ArrayList<>();

    for (String chunk : rawChunks) {
        String cleaned = chunk.trim();
        if (cleaned.length() > 15) { // Only test
meaningful sentences/phrases
            validChunks.add(cleaned);
        }
    }

    if (validChunks.isEmpty() && !text.trim().isEmpty()) {
        validChunks.add(text.trim());
    }

    return new DocData(text, validChunks);
}
}

```